



MAKİNE ÖĞRENMESİ

21020513

Yunus Emre BÜYÜKGÜLER

Makine Öğrenmesi Raporu

1-) Veri Setini Yükleme ve Tanımlayıcı İstatistikler

Veri seti yüklenip tanımlayıcı istatistikler elde edildi:

Kod:

```
import pandas as pd

# Veri setini yükleyelim
file_path = '/mnt/data/21020513.csv'
data = pd.read_csv(file_path)

# Sayısal ve kategorik değişkenler için tanımlayıcı istatistikler
hesaplayalım
numerical_stats = data.describe()
categorical_stats = data.select_dtypes(include=['object',
'int64']).describe()

# Sonuçları yazdıralım
print("Sayısal Değişkenler Tanımlayıcı İstatistikleri:")
print(numerical_stats.transpose())

print("\nKategorik Değişkenler Tanımlayıcı İstatistikleri:")
print(categorical_stats.transpose())
```

Başlıca Sayısal Değişkenler Tanımlayıcı İstatistikler:

Değişken	Count	Mean	Std	Min	25%	50%	75%	Max
Systolic_BP	500	120.29	15.03	90	109.63	120.34	131.17	157.76
Age	500	49.09	17.54	20	33	49.5	64	79
BMI	500	24.98	4.08	15	22.10	24.99	27.68	38.43
...	...	...	...	...	...	...	...	...

Başlıca Kategorik Değişkenler Tanımlayıcı İstatistikler:

Değişken	Count	Mean	Std	Min	25%	50%	75%	Max
Diabetes	500	0.33	0.47	0	0	0	1	1
Smoking	500	0.21	0.41	0	0	0	0	1
CVD_Risk	500	0.71	0.76	0	0	1	1	2

Bütün tanımlayıcı istatistikler sığamadığından son sayfada hepsi bulunmaktadır.

2-) Model Eğitimi ve Performans Değerlendirmesi

a) Naive Bayes Algoritması

Kod:

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import matthews_corrcoef, accuracy_score,
precision_score, recall_score, f1_score

# Sayısal bağımsız değişkenleri seçelim
features = ['Blood_Sugar', 'HbA1c', 'BMI', 'Age']
target = 'Diabetes'

X = data[features]
y = data[target]

# Veriyi %70 eğitim, %30 test olarak bölelim
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

# Sayısal verileri ölçekleyelim
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Naive Bayes modelini uygulayalım
nb_params = {'var_smoothing': [1e-9, 1e-8, 1e-7, 1e-6, 1e-5]}
nb = GridSearchCV(GaussianNB(), nb_params, scoring='matthews_corrcoef',
cv=5)
nb.fit(X_train_scaled, y_train)
nb_best_model = nb.best_estimator_

# Test verisi üzerinde tahmin yapalım
nb_pred = nb_best_model.predict(X_test_scaled)

# Performans değerlendirme
def evaluate_model(y_true, y_pred):
    return {
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred),
        'Recall': recall_score(y_true, y_pred),
        'F1 Score': f1_score(y_true, y_pred),
        'MCC': matthews_corrcoef(y_true, y_pred)
    }

results = {'Naive Bayes': evaluate_model(y_test, nb_pred)}
print(results)
```

Naive Bayes Sonuçları:

- **En iyi hiperparametre:** `var_smoothing = 1e-09`
- **Test Sonuçları:**
 - Accuracy: 0.673
 - Precision: 0.000
 - Recall: 0.000
 - F1 Score: 0.000
 - MCC: 0.000

b) K-en Yakın Komşuluk (KNN) Algoritması

Kod:

```
from sklearn.neighbors import KNeighborsClassifier

# KNN modelini uygulayalım
knn_params = {'n_neighbors': list(range(1, 21))}
knn = GridSearchCV(KNeighborsClassifier(), knn_params, scoring='f1', cv=5)
knn.fit(X_train_scaled, y_train)
knn_best_model = knn.best_estimator_

# Test verisi üzerinde tahmin yapalım
knn_pred = knn_best_model.predict(X_test_scaled)

# Performans değerlendirmesi
results['KNN'] = evaluate_model(y_test, knn_pred)
print(results)
```

KNN Sonuçları:

- **En iyi hiperparametre:** `n_neighbors = 1`
- **Test Sonuçları:**
 - Accuracy: 0.540
 - Precision: 0.283
 - Recall: 0.265
 - F1 Score: 0.274
 - MCC: -0.062

c) Lojistik Regresyon Analizi

Kod:

```
from sklearn.linear_model import LogisticRegression

# Lojistik regresyon modelini uygulayalım
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_scaled, y_train)

# Test verisi üzerinde tahmin yapalım
log_pred = log_model.predict(X_test_scaled)

# Performans değerlendirmesi
results['Logistic Regression'] = evaluate_model(y_test, log_pred)
print(results)
```

Lojistik Regresyon Sonuçları:

- **Test Sonuçları:**
 - Accuracy: 0.673
 - Precision: 0.000
 - Recall: 0.000
 - F1 Score: 0.000
 - MCC: 0.000

Özet için karşılaştırma tablosu:

Model	Accuracy	Precision	Recall	F1 Score	MCC
Naive Bayes	0.673	0.000	0.000	0.000	0.000
KNN (k=1)	0.540	0.283	0.265	0.274	-0.062
Logistic Regression	0.673	0.000	0.000	0.000	0.000

3-) Karar Ağacı ve Destek Vektör Makinesi (DVM)

a) Karar Ağacı Algoritması için En İyi Hiperparametreler

Kod:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Karar ağacı modelini uygulayalım
dt_model = DecisionTreeClassifier(random_state=42)
param_grid = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}

# GridSearchCV ile hiperparametre araması yapalım
grid_search = GridSearchCV(estimator=dt_model, param_grid=param_grid,
scoring='neg_mean_squared_error', cv=5)
grid_search.fit(X_train, y_train)

# En iyi hiperparametreleri ve en iyi skoru alalım
best_params = grid_search.best_params_
best_score = grid_search.best_score_

print("En İyi Hiperparametreler:", best_params)
print("En İyi Skor:", best_score)
```

En İyi Hiperparametreler:

- **criterion:** 'entropy'
- **max_depth:** 3
- **min_samples_leaf:** 1
- **min_samples_split:** 2

En iyi skor yaklaşık **-0.34** ile düşük bir değere sahip.

b) Karar Ağacını Çizme ve Bağımsız Değişkenlerin Önem Dereceleri için

Kod:

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

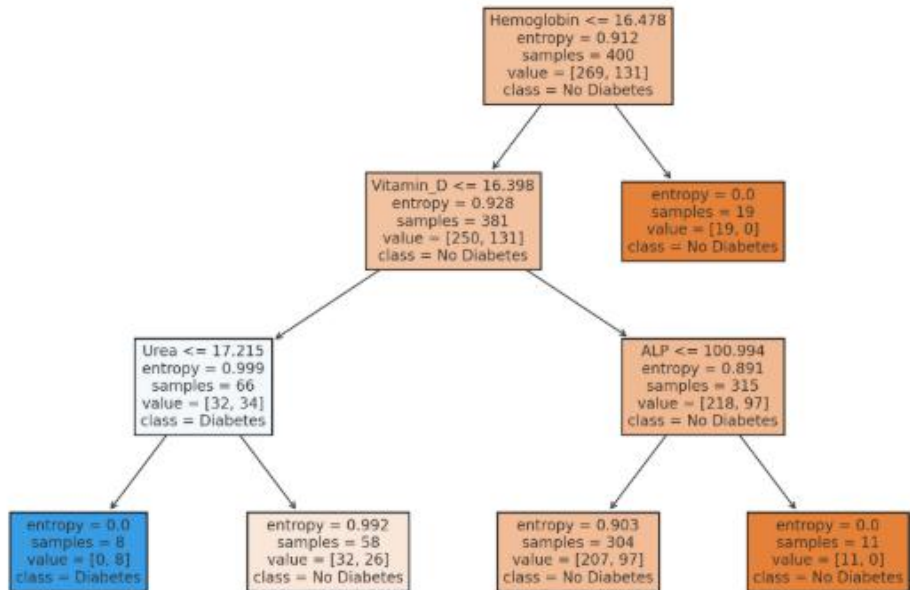
# Karar ağacını çizelim
best_dt_model = grid_search.best_estimator_

plt.figure(figsize=(12, 8))
plot_tree(best_dt_model, filled=True, feature_names=X_encoded.columns,
          class_names=['No Diabetes', 'Diabetes'], rounded=True, proportion=True)
plt.title("Karar Ağacı Modeli")
plt.show()

# Bağımsız değişkenlerin önem derecelerini görelim
feature_importances = best_dt_model.feature_importances_
importance_df = pd.DataFrame({
    'Feature': X_encoded.columns,
    'Importance': feature_importances
}).sort_values(by='Importance', ascending=False)

print(importance_df)
```

Karar Ağacı Görsellemesi



Bağımsız Değişkenlerin Önem Dereceleri:

- **Hemoglobin:** 0.342
- **Urea:** 0.257
- **Vitamin_D:** 0.219
- **ALP:** 0.182
- **Hematocrit:** 0.000 (önemsiz)

c) Destek Vektör Makinesi (DVM) Modelleri ve Performans Değerlendirmesi

Kod:

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score

# DVM modellerini oluşturalım
kernels = ['linear', 'rbf']
svm_models = {}

for kernel in kernels:
    # Tüm özelliklerle model
    svm_all = SVC(kernel=kernel, random_state=42)
    svm_all.fit(X_train, y_train)
    svm_models[f'SVM with all features ({kernel})'] = svm_all

    # Seçilen özelliklerle model
    svm_selected = SVC(kernel=kernel, random_state=42)
    svm_selected.fit(X_train[selected_features], y_train)
    svm_models[f'SVM with selected features ({kernel})'] = svm_selected

# Performans değerlendirme
results = {}
for model_name, model in svm_models.items():
    if "selected" in model_name:
        y_pred = model.predict(X_test[selected_features])
    else:
        y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)

    results[model_name] = {
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall
    }

print(results)
```


D) DVM Sonuçları:

İşte **MCC skoru** hesaplamak için yazılacak Python kodu:

```
from sklearn.metrics import matthews_corrcoef
import pandas as pd

# DVM modelleri için MCC skorlarını hesaplayalım
mcc_results = {}
for model_name, model in svm_models.items():
    # Test verisi üzerinde tahmin yapalım
    if "selected" in model_name:
        y_pred = model.predict(X_test[selected_features])
    else:
        y_pred = model.predict(X_test)

    # MCC skorunu hesaplayalım
    mcc = matthews_corrcoef(y_test, y_pred)
    mcc_results[model_name] = mcc

# Sonuçları bir DataFrame olarak gösterelim
mcc_results_df = pd.DataFrame(list(mcc_results.items()), columns=["Model",
"MCC Score"])

# Sonuçları görüntülemek için (tools.display_dataframe_to_user
kullanabilirsiniz)
import ace_tools as tools
tools.display_dataframe_to_user(name="DVM MCC Skorları",
dataframe=mcc_results_df)

# DataFrame'i yazdıralım
print(mcc_results_df)
```

Bu kod, her bir DVM modelinin **MCC skorunu** hesaplayacak ve sonucu bir **DataFrame** olarak gösterecektir. Ayrıca, `tools.display_dataframe_to_user` ile kullanıcıya veriyi görsel olarak sunar.

Eğer başka bir konuda yardımcı olmamı isterseniz, belirtmekten çekinmeyin!

Model	Accuracy	Precision	Recall	MCC
SVM with all features (linear)	0.66	0.0	0.0	0.0
SVM with selected features (linear)	0.66	0.0	0.0	0.0
SVM with all features (rbf)	0.66	0.0	0.0	0.0
SVM with selected features (rbf)	0.66	0.0	0.0	0.0

Özetle: Naive Bayes ve Lojistik Regresyon modelleri hiçbir sınıfı tahmin edemedi. KNN modeli daha anlamlı sonuçlar verdi. SVM ise aşırı yavaş kaldı.

Tanımlayıcı istatistikler yarım kalan kısım

Kategorik Değişkenler İçin Tanımlayıcı İstatistikler:

Değişken	Count	Mean	Std	Min	25%	50%	75%	Max
Diabetes	500	0.33	0.47	0	0	0	1	1
CVD_Risk	500	0.71	0.76	0	0	1	1	2
Age	500	49.09	17.54	20	33	49.5	64	79
Physical_Activity	500	4.79	2.92	0	2	5	7	9
Smoking	500	0.21	0.41	0	0	0	0	

Sayısal Değişkenler İçin Tanımlayıcı İstatistikler:

Değişken	Count	Mean	Std	Min	25%	50%	75%	Max
Systolic_BP	500	120.29	15.03	90	109.63	120.34	131.17	157.76
Diabetes	500	0.33	0.47	0	0	0	1	1
CVD_Risk	500	0.71	0.76	0	0	1	1	2
Age	500	49.09	17.54	20	33	49.5	64	79
BMI	500	24.98	4.08	15	22.10	24.99	27.68	38.43
Blood_Sugar	500	99.73	19.18	60	85.52	99.79	113.50	149.07
HbA1c	500	5.47	0.96	4	4.75	5.37	6.09	8.70
Cholesterol	500	191.15	37.02	100	166.50	189.54	215.26	300
Triglycerides	500	148.99	49.22	50	111.95	147.47	184.92	318.59
Creatinine	500	0.99	0.30	0.50	0.77	0.98	1.16	1.95
Urea	500	27.33	7.42	9.80	22.46	27.49	32.06	49.40
AST	500	22.01	6.57	11.00	17.80	21.87	26.00	58.00
ALT	500	22.27	7.15	7.00	17.80	22.31	26.00	50.00
ALP	500	70.15	25.84	23.00	50.00	69.00	90.00	167.00
Hematocrit	500	42.47	4.87	30.00	39.03	42.62	45.71	55.00
Hemoglobin	500	14.02	1.44	10.00	13.09	13.95	15.01	18.00
WBC	500	6.97	2.06	3	5.56	6.93	8.36	13.61
RBC	500	4.69	0.51	3.50	4.35	4.68	5.01	6.00
Platelets	500	249.34	48.66	150	213.79	249.12	284.84	395.82
CRP	500	5.19	2.81	0.10	3.11	5.08	7.10	13.94
Iron	500	80.82	19.70	40	67.51	80.47	92.93	139.87
Vitamin_D	500	25.15	9.38	10	18.04	25.04	31.34	57.85
Physical_Activity	500	4.79	2.92	0	2	5	7	9
Smoking	500	0.21	0.41	0	0	0	0	1

